

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 23 September 2015 - Part Theory

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expression R , over the three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$R = ((a b c)^* (c b a)^*)^*$$

Answer the following questions:

- (a) Find a deterministic automaton A equivalent to the regular expression R , and if necessary minimize it.
 - (b) In the minimal automaton A constructed before, project letter c onto the empty string ε , then eliminate the spontaneous transitions, and obtain an automaton B (possibly nondeterministic) that recognizes the projected language.
 - (c) By using some systematic method (not just your intuition), find a regular expression R' for the language $\neg L(B)$, that is, for the complement of the language accepted by the automaton B constructed before.
 - (d) (optional) Say if the original regular expression R is ambiguous or not, and justify your answer.
-

2 Free Grammars and Pushdown Automata 20%

1. Define a free grammar (in *BNF* form) for each of the following three subsets of the Dyck language (with one parenthesis type). Simple solutions are preferred especially for what concerns the number of nonterminal symbols and rules.

- (a) At every nesting level there are exactly two nested parenthesis structures.

Samples:

$$\varepsilon \quad () \left(() () \right) \quad \left(() (() ()) \right) ()$$

Draw the syntax tree for string: $() \left(() () \right)$

- (b) At every nesting level there is an even number of nested parenthesis structures.

Samples:

$$\varepsilon \quad () () () ()$$

Draw the syntax tree for string: $() \left(() () \right)$

- (c) (optional) At every nesting level there is an odd number of nested parenthesis structures.

Samples:

$$() \quad () () () \quad () () \left(() (()) () \right)$$

Counterexamples:

$$\varepsilon \quad () () \quad \left(() () \right)$$

Draw the syntax tree for string: $() \left(() () () \right) ()$

2. Consider a language of integer arithmetic expressions that features the following structures and operations:

- The integer variables are symbolized by the one terminal v .
- The integer constants are sequences of one or more decimal digits $0, \dots, 9$, not starting with one or more digits 0 .
- There are the operations of addition $+$ and multiplication $\times$, and their precedence is as usual (multiplication has a priority higher than addition does).
- There are the operations of factorial $(a)!$ and of power $(b) * (e)$, e.g., $(v) * (2)$ means v^2 in traditional notation. The elements a, b and e may be an atom or a whole expression, and they are always enclosed in parentheses. Valid examples:

$$(v)! \quad (2 \times v + 1)! \quad (v) * (v) \quad (v) * (v + 1) \quad (2 \times v) * (v)$$

- Since the numerical value of the factorial and power expressions grows very quickly, it is forbidden that the argument a of a factorial and the exponent e of a power contain more factorials or powers. For instance, these expressions are forbidden:

$$((v)!)! \quad (1 + (v) * (v))! \quad (2) * ((v)!) \quad (2) * ((v) * (v))$$

but these are permitted (because factorials and powers may occur in the base of any power):

$$((v) * (v)) * (2) \quad ((v)!) * (2)$$

- The factorial and power operations have a priority higher than addition and multiplication do.
- In addition to those mandatory for the factorial and power operations, there are parentheses (round open “(” and closed “)”), which can be used to change the conventional precedence of the various operations.

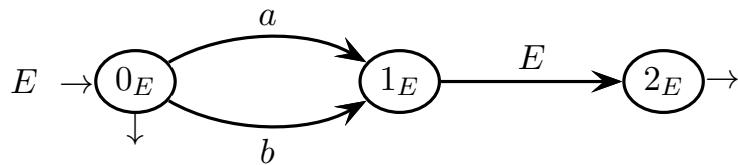
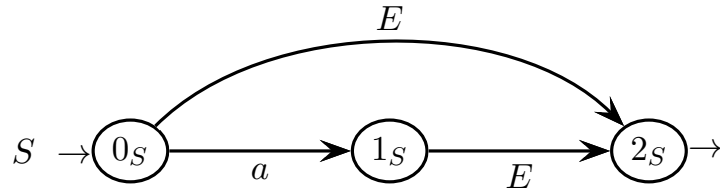
Here is a longer valid expression:

$$v + 2 \times (v + 1) * (v + v) + (2 + v \times v)!$$

Write an extended (*EBNF*) grammar, not ambiguous, that generates the language sketched above, and draw the syntax tree of the sample long expression above.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar below, over the two-letter alphabet $\Sigma = \{ a, b \}$, represented as a machine net (axiom S):



Answer the following questions:

- (a) Draw the complete *PCFG* of the grammar, say if the grammar is of type *ELL*(1), or at least of type *ELL*(k) for some $k \geq 2$, and justify your answer.
- (b) Draw the complete pilot of the grammar, say if the grammar is of type *ELR*(1) and justify your answer (list all the conflicts if any).
- (c) Analyze the valid sample string $ab a$ by means of the Earley method, and show on the Earley vector why such a string is accepted. Furthermore, show on the vector each prefix of the sample string that would be accepted as well.
- (d) (optional) Say if the language can be represented by a deterministic free grammar, and justify your answer.

Earley vector of string $a\ b\ a$ (to be filled)
 (the number of rows is not significant)

0	a	1	b	2	a	3

4 Language Translation and Semantic Analysis 20%

1. Consider the following translation τ , with identical source and destination alphabets $\Sigma = \Delta = \{ a, b \}$ of two letters:

$$\tau (a^h b^k) = b^k a^h \quad \text{with } h \geq k \geq 0$$

For instance $\tau (a^3 b^2) = b^2 a^3$. Translation τ acts as a sort of commutator.

Answer the following questions:

- (a) Write a *BNF* syntax translation scheme (or a *BNF* translation grammar) that computes translation τ , and briefly justify its correctness.
 - (b) Draw the source and destination syntax trees of the sample translation shown above.
 - (c) (optional) Say if the syntax transduction scheme found before is deterministic or not, and justify your answer. In the case it is not, argue if the translation τ is deterministic or not.
-

2. Consider the following abstract syntactic support for a language of arithmetic expressions with addition and parentheses, where the atomic addends are symbolized by a terminal a (axiom S).

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow E + E \\ E &\rightarrow (E) \\ E &\rightarrow a \end{aligned}$$

For instance, a possible expression is $a + (a + a) + a$.

Starting from this syntactic support, define an attribute grammar that answers the following questions (use the various tables and trees prepared on the next pages):

- (a) The *nesting level* of an addend is the number of parenthesis pairs around the addend. By using a synthesized attribute h , compute the *height* of the whole expression, i.e., 1 plus the maximal nesting level of all the atomic addends. In the sample expression $a + (a + a) + a$, the value of h is 2. Decorate the syntax tree of the sample expression above with the values of h .
- (b) For each atomic addend a , compute the attribute nl that expresses the nesting level of a (see the definition above). The minimal value of nl is 0.

For instance, in the above sample expression, if we represent nl as a subscript of each atomic addend, i.e., a_{nl} , then its values can be visualized as follows:

$$a_0 + (a_1 + a_1) + a_0$$

Decorate the syntax tree of the sample expression above with the values of nl .

- (c) (optional) For each atomic addend a , compute the attribute p that expresses the position (from the left) of a in the list of addends (atomic or not) at the same nesting level as a in a (sub)expression. The minimal value of p is 0.

For instance, in the above sample expression, if we represent p as a subscript of each atomic addend, i.e., a_p , then its values can be visualized as follows:

$$a_0 + (a_0 + a_1) + a_2$$

Decorate the syntax tree of the sample expression above with the values of p and of any other auxiliary attribute possibly used to compute p .

Hint: we suggest to use an auxiliary attribute na , of synthesized type, to count the number of addends at the same nesting level in a subexpression; the examinee may however choose any other solution (s)he prefers.

attributes assigned to be used (complete the specification)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>(non)term.</i>	<i>meaning</i>
left	<i>h</i>	integer ≥ 1		height of the whole expression
	<i>nl</i>	integer ≥ 0		number of parenthesis pairs around an addend
	<i>p</i>	integer ≥ 0		position of an addend in the list of addends at the same nesting level in a (sub)expression

attributes to be added, if any (complete the specification)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>(non)term.</i>	<i>meaning</i>
left	<i>na</i>	integer ≥ 1		number of addends at the same nesting level in a (sub)expression

Question (a):

syntax

semantics - question (a)

$$1: \quad S_0 \rightarrow E_1$$

$$2: \quad E_0 \rightarrow E_1 + E_2$$

$$3: \quad E_0 \rightarrow (E_1)$$

$$4: \quad E_0 \rightarrow a$$

Question (b):

syntax

semantics - question (b)

1: $S_0 \rightarrow E_1$

2: $E_0 \rightarrow E_1 + E_2$

3: $E_0 \rightarrow (E_1)$

4: $E_0 \rightarrow a$

Question (c):

syntax

semantics - question (c)

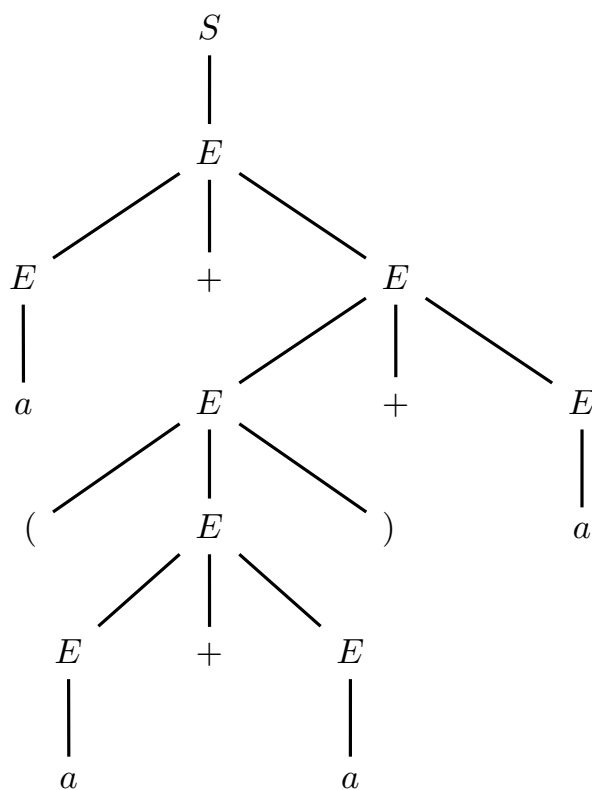
$$1: \quad S_0 \rightarrow E_1$$

$$2: \quad E_0 \rightarrow E_1 + E_2$$

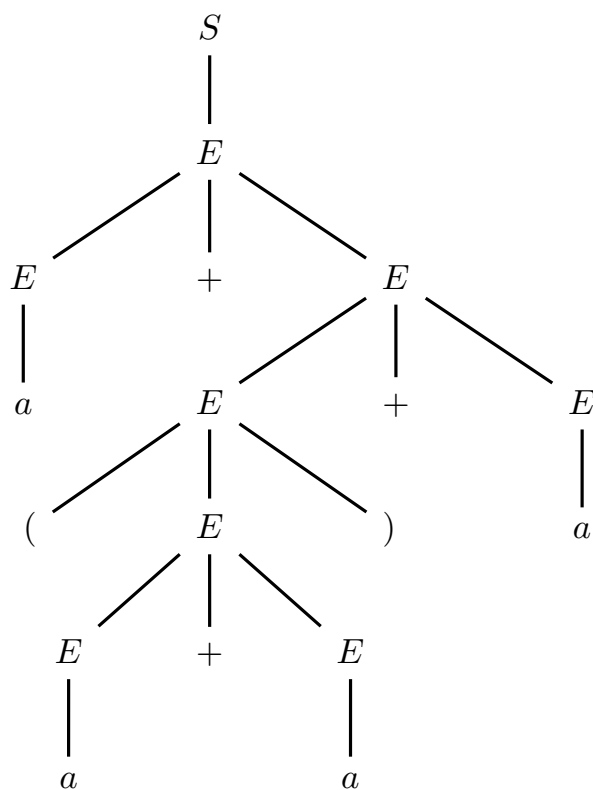
$$3: \quad E_0 \rightarrow (E_1)$$

$$4: \quad E_0 \rightarrow a$$

question (a)



question (b)



question (c)

